# RDTII Rocky

Setup guide: getting the tool running on a computer that has nothing installed

Team: Rocky has a home run

6 October 2026

## Before you start

**Everything here starts from the terminal**, the black window where commands are typed. Open it first and keep it open: every command below is pasted into it.

- **Windows**: press the Start button, type `PowerShell`, open *Windows PowerShell*.
- **Mac**: press `Cmd`+`Space`, type `Terminal`, press Enter.

This guide takes a computer with nothing on it and leaves you at the tool's own instructions (the README's *Verify* step). You do not need to be a programmer. You will type commands into one black window; every command below can be copied and pasted whole.

**What you are setting up.** The tool has three stages, and this guide installs what each one needs, in that order:

| Stage | What it does | Needs |
|---|---|---|
| 1. Scraping | collects an economy's laws from its official portals | Python packages + a browser engine |
| 2. Extraction | splits the laws into numbered provisions | Python packages + Tesseract (for scans) |
| 3. Mapping | maps each provision to the RDTII indicators | Python packages + Ollama (free engine) |

**Time and space.** About 30 minutes of typing and waiting, 2 GB of disk. A further 20 minutes and 9 GB for the free local AI engine (Section 7), and 2 GB for the meaning index (Section 6); both are optional. Everything runs on your own computer; nothing is uploaded anywhere.

**How to read this guide.** Each step first *checks* whether something is already on the computer. If the window answers with a version number, skip the install and go to the next step. If it says *not found*, *not recognized* or *command not found*, run the install box underneath.

**When to open a new window.** A window opened before a program was installed does not know that program exists. So after installing a *program* (Python, Git, Tesseract, Ollama), close the window, open a new one and go back to the project folder as in Section 1. The *packages* installed with `pip` (Sections 4 to 6) need no new window: they go into the project's own folder, where every command finds them at once. The tool itself looks for packages only when it starts, so after the meaning index of Section 6 restart the tool, not the window. Each step says which applies.

# 1 Get the project onto the computer

Download the project as a ZIP from the repository page on GitHub (green **Code** button → **Download ZIP**), then unzip it wherever you like to keep it; the Desktop is the example used below. GitHub names the unzipped folder after the branch or tag it came from: `rdtii-rocky-next-main` or `rdtii-rocky-next-final-submission`. Rename it to `rdtii-rocky-next` (right-click the folder, *Rename*) so that the commands below find it. You should end up with a folder called `rdtii-rocky-next` holding folders named `interface`, `stages` and `submission`.

Now point the window at that folder. Every later command assumes you are inside it. If the folder is not on your Desktop, change the path in the command to where you put it. The easy way: type `cd` and a space, drag the folder from its file window into the black window (its path appears there), then press Enter.

**Windows** (the folder on the Desktop):

```
cd $HOME\Desktop\rdtii-rocky-next
```

**Mac** (the folder on the Desktop):

```
cd ~/Desktop/rdtii-rocky-next
```

Check you are in the right place — this lists the project's files:

```
dir        (Windows)
ls         (Mac)
```

*If you prefer the developer's way:* if `git` is already installed, these three lines do the same, without the rename; run them in the folder where the project should live. The ZIP above needs no extra program.

```
git clone https://github.com/JohnChen-kmg/rdtii-rocky-next
cd rdtii-rocky-next
git checkout final-submission
```

If there is no `git` (the check `git -version` answers *not found*), either stay with the ZIP above, which needs no extra program, or install it; it is a program, so open a new window afterwards.

**Windows**:

```
winget install Git.Git
```

**Mac** (a dialog opens; press Install):

```
xcode-select --install
```

## 2 Python: check, then install if missing

Python is the language the tool is written in. Check first.

**Windows**:

```
python --version
```

**Mac**:

```
python3 --version
```

**Good:** it answers `Python 3.12.x` or higher. Go to Section 3.

**Not good:** it says `not recognized` / `command not found`, it opens the Microsoft Store, or the number is below 3.12. Install Python:

1. Open https://www.python.org/downloads/ and download the latest 3.12 or newer.
2. **Windows**: in the installer, **tick "Add python.exe to PATH"** on the first screen before pressing Install. This one tick is what makes the command above work.
3. **Mac**: open the downloaded `.pkg` and accept every default.
4. **Close the window and open a new one** (it only notices the new Python then), go back to your project folder as in Section 1, and run the check again.

## 3 Give the project its own package area

Rather than installing packages into your whole computer, the tool keeps them in one folder inside the project, called `.venv`. This is one command and takes a few seconds.

**Windows**:

```
python -m venv .venv
```

**Mac**:

```
python3 -m venv .venv
```

Nothing is printed when it works. From here on, commands say `.venv\Scripts\` (Windows) or `.venv/bin/` (Mac) so that the project's own Python is used.

# 4 Stage 1 — Scraping: the packages that fetch the laws

## Check

**Windows**:

```
.venv\Scripts\python -c "import playwright, requests, tenacity, dateutil, yaml,
    jsonschema, cryptography; print('Stage 1 ready')"
```

**Mac**:

```
.venv/bin/python -c "import playwright, requests, tenacity, dateutil, yaml, jsonschema,
    cryptography; print('Stage 1 ready')"
```

**Good:** it prints `Stage 1 ready`. **Not good:** it prints `ModuleNotFoundError: No module named '...'`. Install below.

## Install

One command installs the packages of all three stages (they come from one list), so you only run this once:

**Windows**:

```
.venv\Scripts\pip install -r requirements-demo.txt
```

**Mac**:

```
.venv/bin/pip install -r requirements-demo.txt
```

It prints a long list and ends with `Successfully installed ...`. Allow 5 minutes. No new window is needed after `pip`: the packages go into the project folder, where every later command finds them at once.

Stage 1 also needs a browser engine to read portals that build their pages with JavaScript. It is about 150 MB:

**Windows**:

```
.venv\Scripts\python -m playwright install chromium
```

**Mac**:

```
.venv/bin/python -m playwright install chromium
```

Then run the *Check* box again; it should now print `Stage 1 ready`.

### What these are for

| Package | Why Stage 1 needs it |
|---|---|
| `playwright` (+ Chromium) | drives a real browser for portals that need one |
| `requests`, `httpx` | downloads a document over the web |
| `tenacity` | waits and retries politely when a portal is busy |
| `python-dateutil` | reads the dates portals print in their own styles |
| `PyYAML` | reads the list of portals for each economy |
| `jsonschema` | checks the record of what was collected before handing it on |
| `cryptography` | Laws of Malaysia serves its listings encrypted |

# 5 Stage 2 — Extraction: reading documents, including scans

## Check

**Windows**:

```
.venv\Scripts\python -c "import rdtii_p2.cli, pypdfium2, pytesseract, pydantic; print('
    Stage 2 ready')"
```

**Mac**:

```
.venv/bin/python -c "import rdtii_p2.cli, pypdfium2, pytesseract, pydantic; print('Stage
     2 ready')"
```

## Install

The extraction stage is part of the project, so it is installed from the project folder itself:

**Windows**:

```
.venv\Scripts\pip install -e stages/p2-extract
```

**Mac**:

```
.venv/bin/pip install -e stages/p2-extract
```

## Tesseract — only for scanned documents

Lao PDR's laws are 96% scans, so Stage 2 reads them with OCR. Check:

```
tesseract --version
```

**Good:** it prints `tesseract 5.x`. **Not good:** *not found*. Install:

**Windows**:

```
winget install UB-Mannheim.TesseractOCR
```

**Mac** (needs Homebrew; if `brew` is not found, install it from https://brew.sh first):

```
brew install tesseract
```

Close the window and open a new one, then check again. The language packs (English, Lao, Chinese, Portuguese) are already inside the project — nothing to download.

### What these are for

| Package | Why Stage 2 needs it |
|---|---|
| `pypdfium2` | opens a PDF and takes out its text, or a picture of the page |
| `pytesseract` + Tesseract | reads a scanned page into text (OCR) |
| `Pillow` | cleans up the page image before OCR |
| `beautifulsoup4`, `lxml`, `selectolax` | read a law published as a web page |
| `pydantic` | checks each provision has the fields the next stage expects |
| `jiwer` | measures how accurate the OCR was |

# 6 Stage 3 — Mapping: matching provisions to indicators

### Check

**Windows**:

```
.venv\Scripts\python -c "import numpy, bm25s, pandas, openpyxl, anthropic, ollama; print
    ('Stage 3 ready')"
```

**Mac**:

```
.venv/bin/python -c "import numpy, bm25s, pandas, openpyxl, anthropic, ollama; print('
    Stage 3 ready')"
```

These came with the one `pip install` of Section 4. If anything is missing, run that command again.

### What these are for

| Package | Why Stage 3 needs it |
|---|---|
| `bm25s`, `PyStemmer` | finds the provisions worth reading, by keyword, at no cost |
| `numpy` | the arithmetic behind that search |
| `anthropic` | talks to the hosted engine (Engine A), if you choose it |
| `ollama` | talks to the free local engine (Engine B) |
| `pandas` | builds the per-economy reports |
| `openpyxl` | writes the Excel evidence workbook |

### The meaning index — optional, needed for Chinese and Lao

Before an engine reads anything, Mapping has to pick, out of thousands of provisions, the few worth reading for each indicator. The tool does this in two ways. The *keyword index* looks for the indicator's words; it is built in a minute and needs nothing beyond the packages of Section 4. The *meaning index* compares what a provision says with what an indicator asks, whatever words and whatever language are used, so it also finds the right provisions in Chinese or Lao laws, where keywords alone find almost nothing; the score thresholds, the tool's default selection rule, are read on it. It needs two more packages and, the first time a run builds it, a model download of about 2 GB; building it takes a while on a computer without a graphics card.

**You can skip it** for English laws: on the Mapping page set **Selection** to **Caps** and **Meaning index** to **Skip**, and the tool uses the keyword index alone. For Chinese or Lao laws, or to use the score thresholds, install it:

**Windows**:

```
.venv\Scripts\pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv\Scripts\pip install sentence-transformers
```

**Mac**:

```
.venv/bin/pip install torch --index-url https://download.pytorch.org/whl/cpu
.venv/bin/pip install sentence-transformers
```

If the tool is already open, close it and start it again: it looks for packages only when it starts. The model download happens by itself during the first run that builds the index.

## 7 The free local engine (optional, but it costs nothing)

The tool declares two AI engines. **Engine A** is hosted (Claude) and needs a key you pay for. **Engine B** runs on your own computer and is free. For Engine B, check:

```
ollama --version
```

If it is not found, install Ollama from https://ollama.com (a normal installer for Windows and Mac). Then close the window, open a new one, go back to the project folder and download the model the tool declares — about 9 GB, so allow time:

```
ollama pull qwen2.5:14b
```

Confirm it is the exact model we declared (the digest should start `sha256:7cdf5a0187d5`):

```
ollama list --digests
```

**You can skip this section** and still open the tool, read every filed result, review rows and export them. You need an engine only to start a *new* mapping run.

# 8 Keys, settings and configuration files

There is nothing to configure. No file to edit, no `.env`, no environment variable to set: every location has a working default inside the project folder.

If you want to use the hosted engine (Engine A), you type its key *into the tool* — Mapping tab, **Engine API keys** at the top of the page. The key is held in memory for as long as the tool is open and is never written to disk, never shown again, and never sent anywhere except to that provider.

## 9 Start the tool

- **Windows**: double-click `Start-RDTII-Rocky.cmd` in the project folder.
- **Mac**: double-click `Start-RDTII-Rocky.command`.

The pages open in a window of their own, using the Edge or Chrome already on your machine. Closing that window stops the tool.

A **Mac** may need one extra step, once: if the double-click opens the file as text, or says it cannot be executed, the unzip has dropped its permission to run. Give it back from the window, inside the project folder, then double-click again:

```
chmod +x Start-RDTII-Rocky.command
```

If the double-click does nothing, start it from the window instead:

**Windows**:

```
.venv\Scripts\python interface/app.py
```

**Mac**:

```
.venv/bin/python interface/app.py
```

. . . then open http://127.0.0.1:8765/ in your browser.

## 10 What a successful setup looks like

You are ready for the tool's own instructions when all of this is true:

1. The three check commands print `Stage 1 ready`, `Stage 2 ready`, `Stage 3 ready`.
2. The tool opens and shows the **Overview** page.
3. Along the top of the page, the health dots read: **Stages** green; **Tesseract** green; **Chromium** green; **Ollama** green if you did Section 7, grey if you skipped it; **Key** grey until you type one, which is correct.
4. Mapping tab → **3.3 Output** lists the finale's run (52 rows, six economies). Clicking a row shows the original law text beside its English, with the source link.
5. Appendix tab → **Check the run layer** ends with **Finished**.

If all five hold, go to the README's *Verify* section and carry on there.

## 11 If the window says something is not found

| What you see | What to do |
|---|---|
| `python is not recognized` (Windows) | Python is missing or was installed without the PATH tick. Re-run the installer, tick *Add python.exe to PATH*, open a new window. |
| Typing `python` opens the Microsoft Store | Same answer: install from python.org with the PATH tick. |
| `ModuleNotFoundError: No module named 'x'` | A package is missing. Re-run `pip install -r requirements-demo.txt` from Section 4, inside the project folder. |
| `No such file or directory: requirements-demo.txt` | You are in the wrong folder. Repeat the `cd` command of Section 1. |
| `tesseract: command not found` | Section 5. On Mac you need Homebrew first. Open a new window afterwards. |
| The Ollama dot stays red | Ollama is installed but not running: open the Ollama app once, then refresh the page. |
| The page does not open | Start the tool from the window (Section 9) — the message there says what is wrong. |
| `The meaning index needs torch and sentence-transformers` | Section 6: install the two packages, or set **Meaning index** to **Skip** for English laws. |
| A stage's **Start** button stays greyed out | Press **Check** on that page first. The failed line tells you exactly what is missing. |

---

Questions: John (Jiaxiang) Chen, jiaxiangchen.kmg@gmail.com. The tool is Apache 2.0 licensed; this guide covers the release tagged `final-submission`.